

Self-Hosted LLM Serving Platform

A 90-second demo, and two things the measurements turned up.

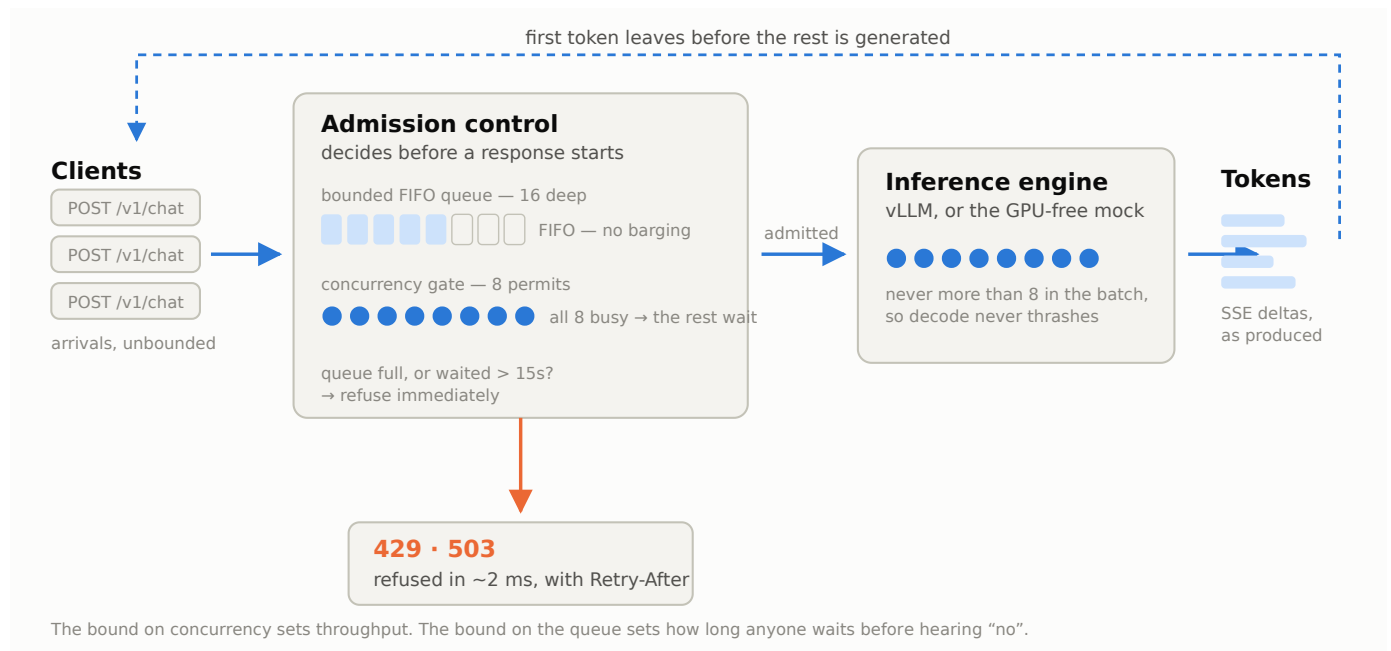
Prepared 13 September 2026 · all figures measured, seed 7, 16.0s per level (4.0s warm-up discarded) · 8 concurrent slots over a 16-deep queue

1 The 90 seconds

Time	Show	Say
0:00-0:15	Architecture diagram	A request arrives. Before anything expensive happens, admission control decides whether the server can take it. Free slot → the engine, and tokens stream back. All eight busy → a short wait in a bounded queue. Queue full → refused immediately, in about two milliseconds.
0:15-0:30	One request, streaming	Normal case first. The response streams token by token — the first token arrives long before the last one is generated.
0:30-1:00	Ramp past capacity (live)	Traffic ramps 4 → 8 → 16 → 32 req/s against the real server, one row per second. At the first orange row the queue is full and excess is refused. The successful requests keep completing at the same rate. Nothing times out.
1:00-1:20	The chart	Sixteen times the offered load. Goodput is the flat line; everything above it is shed. Zero failures. Inter-token latency is flat — whoever got in gets the same experience at 32 req/s as at two.
1:20-1:30	Takeaway	Two bounds, two jobs. Concurrency sets throughput. The queue sets how long someone waits before hearing “no”. Without them that orange line is a backlog instead — work piling up for clients who already gave up.

Run `.\demo\run_demo.ps1` . It pauses before each beat, so you can start recording and get a clean take. The emotional beat is the first orange row in the ramp — pause there.

2 The picture (0:00-0:15)



3 What was measured (1:00-1:20)

Goodput across a 16× range of offered load

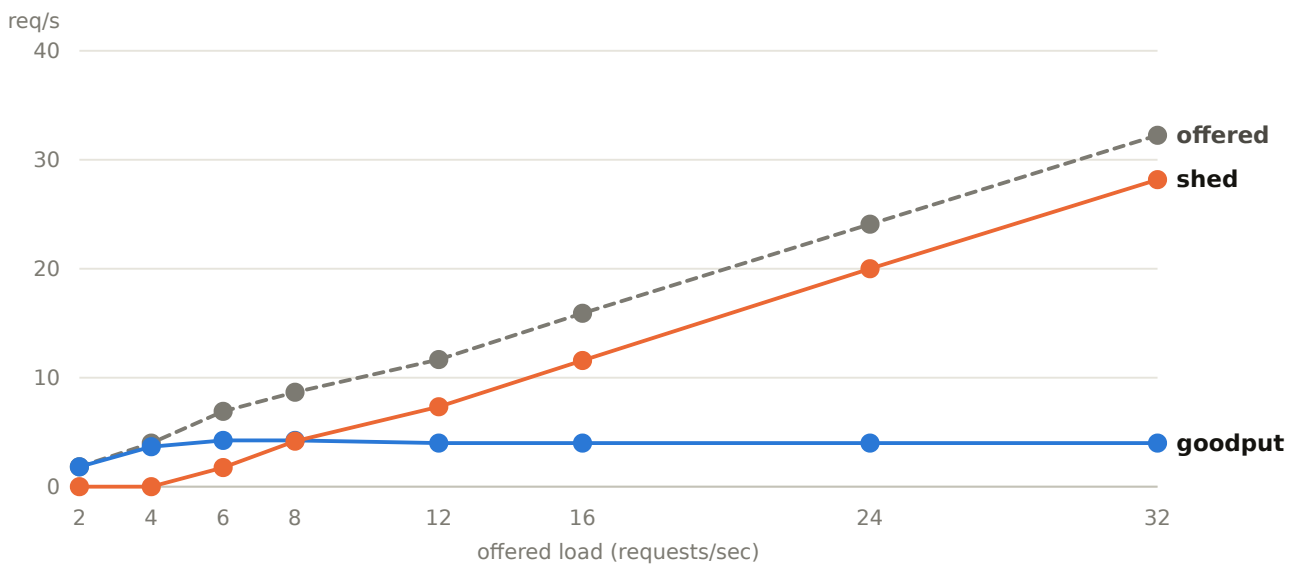
4.0-4.3 req/s

0 failed requests. Everything above capacity was refused in milliseconds rather than queued indefinitely.

— offered — goodput (succeeded) — shed (429/503)

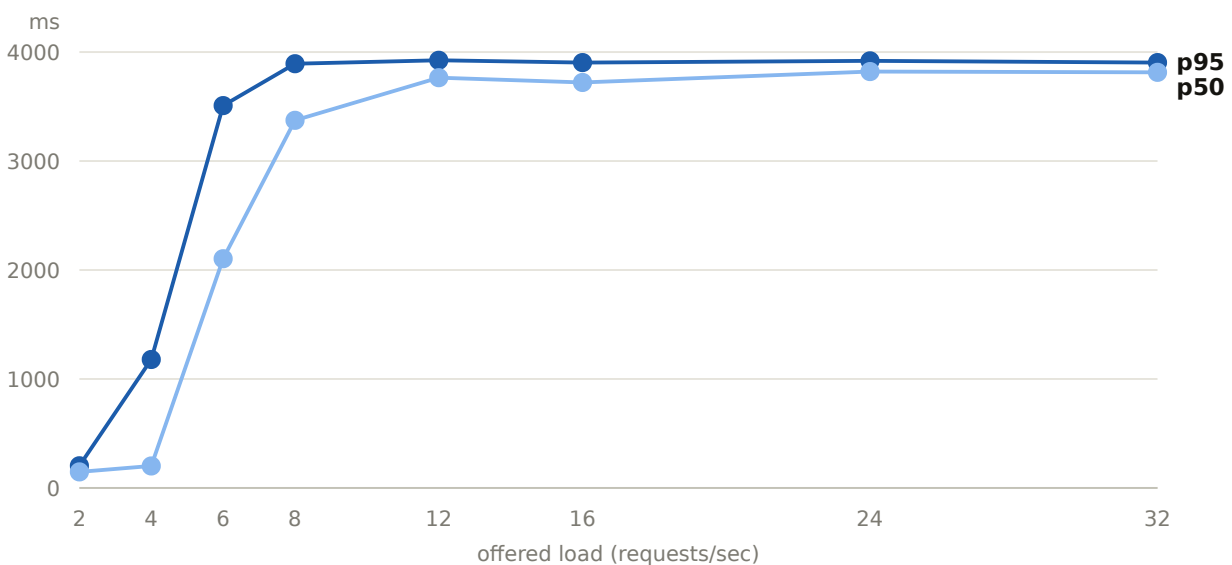
Goodput holds flat while the excess is shed

Offered load rises 2→32 req/s. Successful throughput does not move.



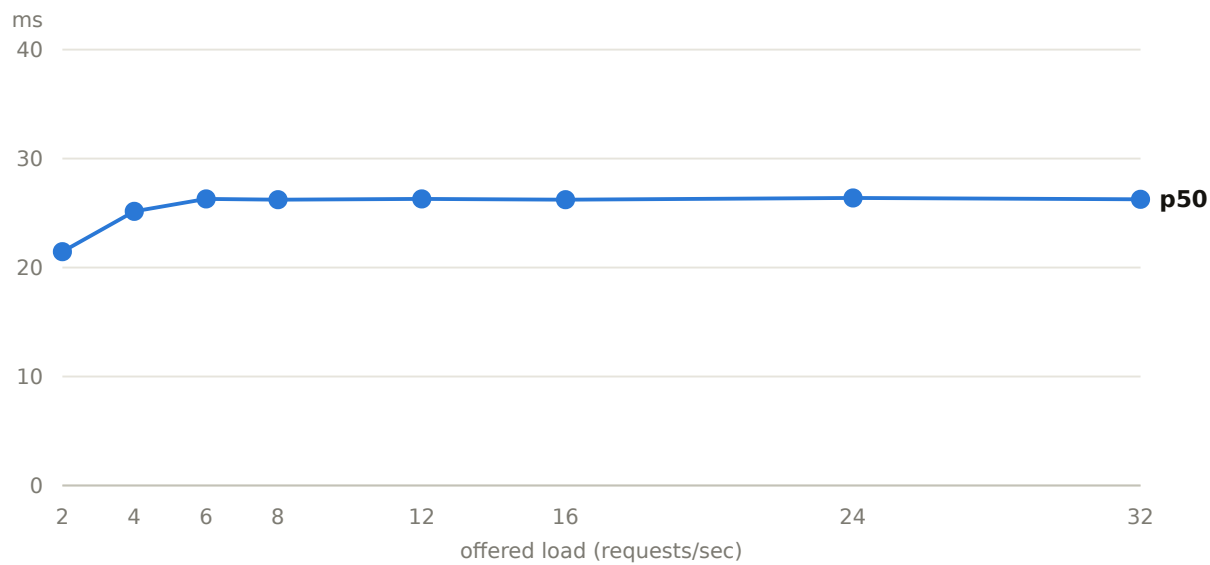
Queue wait is what degrades — and it is bounded

Time to first token, measured from arrival, so queue wait is inside it.



Tokens keep flowing at the same pace for whoever got in

Inter-token latency, p50. Bounded concurrency means the engine is never oversubscribed.



offered	arrived/s	goodput/s	shed/s	shed %	TTFT p50	TTFT p95	ITL p50
2	1.8	1.83	0.0	0%	146	204	21
4	4.0	3.67	0.0	0%	203	1179	25
6	6.9	4.25	1.8	29%	2103	3510	26
8	8.7	4.25	4.2	50%	3373	3893	26
12	11.7	4.00	7.3	65%	3765	3925	26
16	15.9	4.00	11.6	74%	3720	3903	26
24	24.1	4.00	20.0	83%	3821	3919	26
32	32.2	4.00	28.2	88%	3814	3904	26

4 Two findings

The load harness mis-measures under overload

Two independent faults in `loadtest/harness.py`, both of which deflate reported rates:

1. `_open_loop` under-offers. It chains relative sleeps — `create_task(...)` then `await asyncio.sleep(expovariate(rps))` — so each iteration inherits the scheduling overshoot of every previous one. Once the event loop is busy that compounds: a 32 req/s target delivered about 12. Arrival times need to be absolute, so a late wake-up is caught up on rather than carried forward.

2. `run_phase` 's window includes the drain. The clock starts before the arrivals and stops after `gather()` has waited for every straggler, so a 16-second arrival window can be divided by roughly 26 seconds of wall clock. Every derived rate — `throughput_rps`, `goodput_rps`, `output_tps` — is deflated, by about 60% at these levels.

This is the tool behind the README's headline numbers. The *relative* story is unaffected (numerator and denominator move together), but the absolute rates are not trustworthy under overload. Both are fixable without changing the harness interface; `demo/measure_inprocess.py` shows the corrected shape of each.

Queue depth is a latency knob, not a capacity knob

Swept at fixed concurrency (8 slots), 32 req/s offered:

<code>max_queue_size</code>	<code>goodput</code>	<code>shed</code>	<code>TTFT p95</code>
8	4.3 req/s	82%	2.0 s
16	4.3 req/s	79%	3.9 s
64 — current default	4.3 req/s	64%	15.0 s

Goodput is identical across all three. A deeper queue buys no throughput whatsoever; it only converts a fast refusal into a long wait. At a ~4 req/s drain rate the shipped default of 64 is a ~15-second queue — which is also exactly `queue_timeout_s`, so the queue can only ever be fully drained at the timeout boundary. The demo runs at 16 and says so.

Two decisions. Whether to fix the harness — contained, but it changes numbers the README quotes. And whether 64 is the right shipped default for `max_queue_size`.

Figures produced by `demo/measure_inprocess.py`, driving the real `AdmissionController` and `MockEngine` in-process. The HTTP layer is not exercised in the sweep; beat 2 of the demo uses the real server, so the 429s on screen are genuine wire responses.